Концепции машинного обучения

Классификация / Decision Tree / Random Forest /
Gradient Boosting / SVM / KNN



Задача классификации

На выходе — один из заранее заданных классов.

$$a(x) \in \{0, 1\}$$

Бинарная классификация = два возможных класса.

Линейный score

$$f(x) = w^T x + b$$

x — признаки
w — веса
b — сдвиг

$$margin = y \cdot f(x)$$

Чем больше положительный margin, тем увереннее правильная классификация.

Titanic как ML-задача

Маленький датасет, но почти все типовые проблемы табличного ML здесь есть.

891

пассажир

38.4%

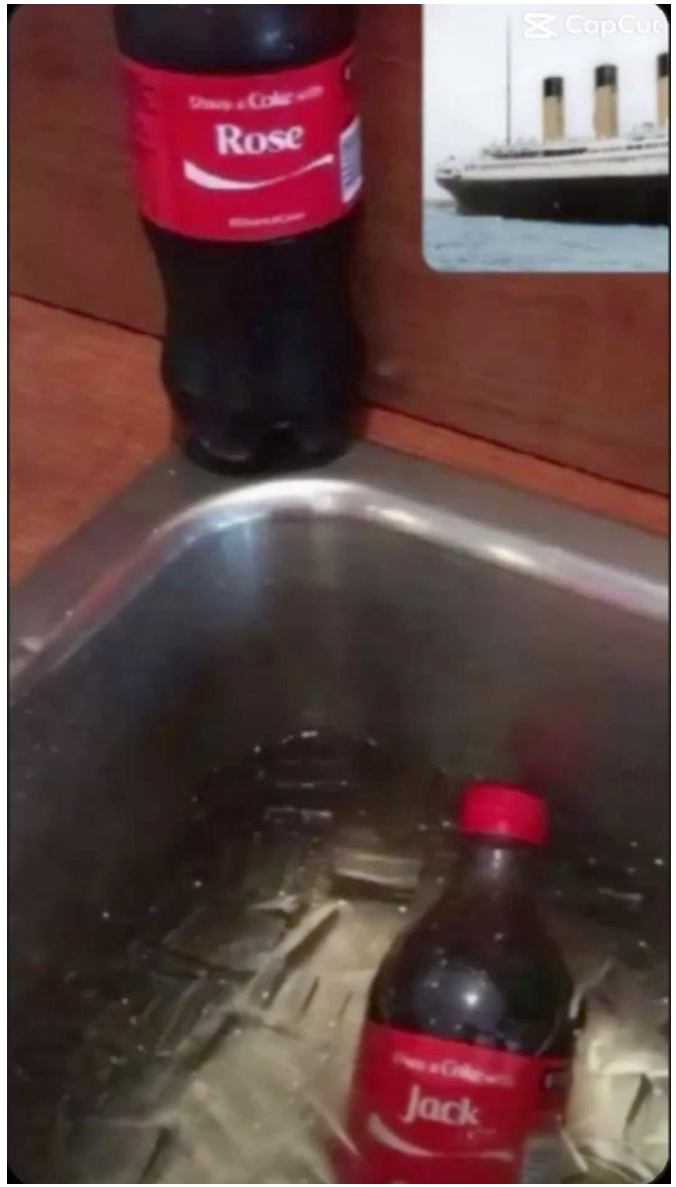
Survived = 1

12

исходных колонок

3

типа проблем



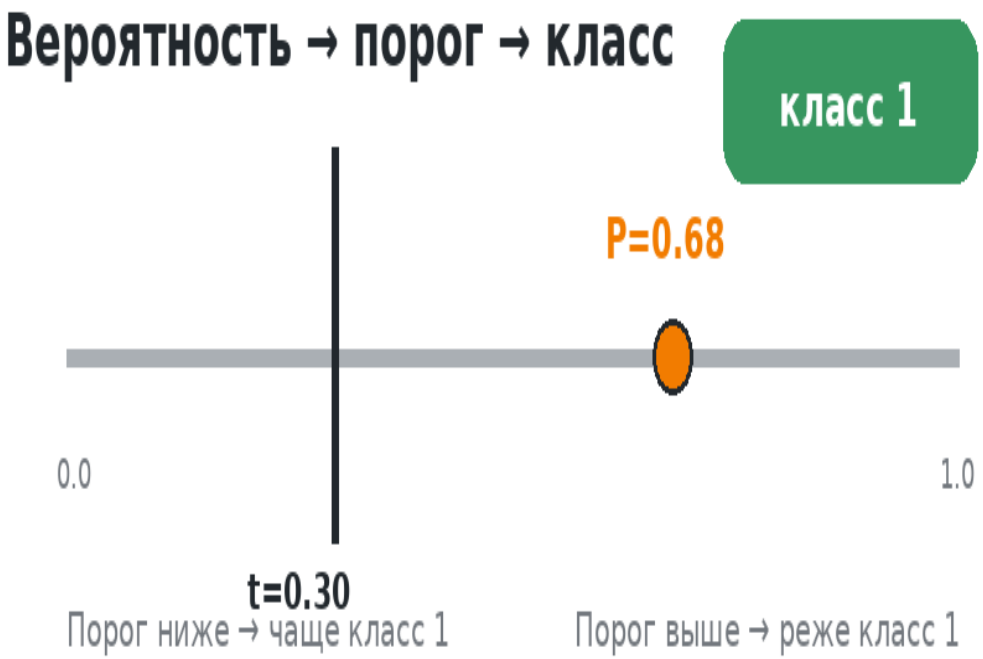
Зачем нужен порог?

Порог превращает непрерывный score модели в конечный класс.

f(x) ≥ t → класс 1

f(x) < t → класс 0

- Для probability часто стартуют с t = 0.5.
- Порог можно выбирать под метрику или стоимость FP/FN.
- Порог — часть решения, а не “константа природы”.



Классификация ≠ «выдать 0 или 1»

Сначала модель строит score / вероятность, затем порог переводит её в класс.

X -> модель -> score / P(y=1|x) -> порог -> класс

Пример: P(survived)=0.68

t = 0.5 → класс 1

Порог ниже ↑ Recall · обычно ↑ FP

Порог выше ↑ Precision · обычно ↓ Recall

Accuracy

Доля всех правильных ответов. Удобная, но опасная при дисбалансе классов.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Считаем положительный и отрицательный классы одинаково.

Precision

Смотрим только на объекты, которые модель назвала классом 1.

$$\text{Precision} = TP / (TP + FP)$$

Если модель сказала «1», насколько этому прогнозу можно верить?

Recall

Смотрим на все истинные объекты класса 1 и считаем, сколько модель не пропустила.

$$\text{Recall} = TP / (TP + FN)$$

Чем выше Recall, тем меньше положительных объектов мы пропускаем.

F1-score

Гармоническое среднее Precision и Recall.

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

Если Precision или Recall близка к нулю, F1 тоже будет низкой.

Ловушка Accuracy

На Titanic можно получить ≈62% accuracy, не построив никакой модели.

61.5%

если всегда предсказывать 0

Recall класса “выжил” = 0

Модель не нашла ни одного выжившего.



Accuracy, Precision, Recall, F1

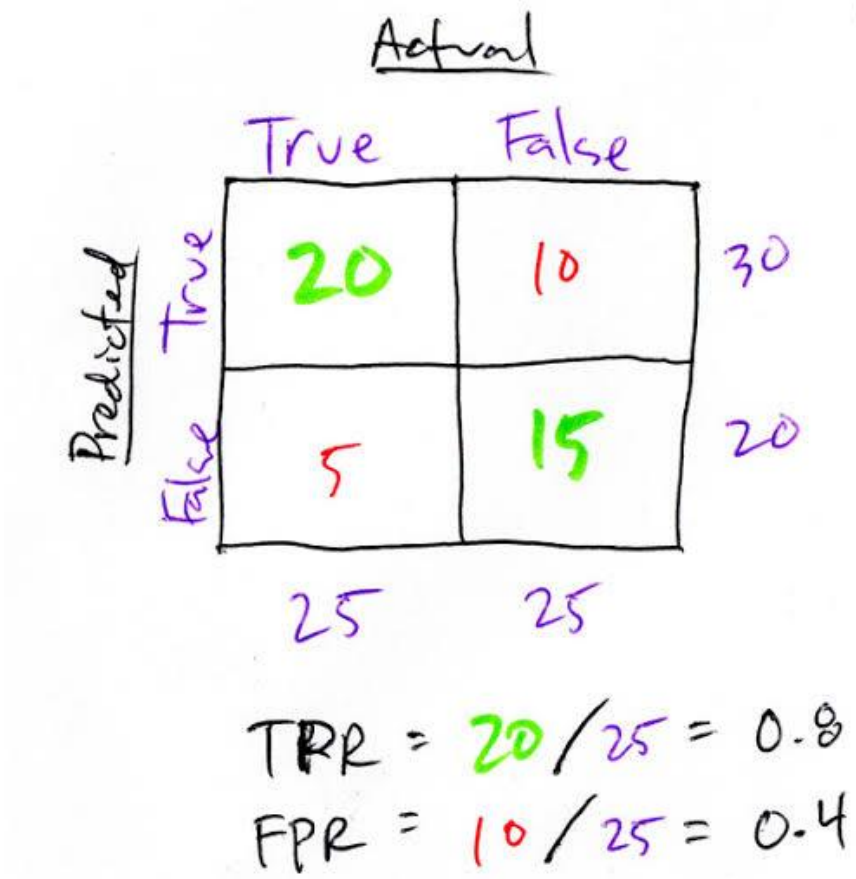
Метрика должна отвечать на вопрос, какой тип ошибки для нас важен.

Accuracy	**(TP + TN) / N**	доля всех правильных ответов
Precision	**TP / (TP + FP)**	насколько верить положительному прогнозу
Recall	**TP / (TP + FN)**	сколько положительных объектов нашли
F1	**2PR / (P + R)**	баланс Precision и Recall

В лабораторной основная метрика — F1 > 0,70

Confusion Matrix

Классификация — не одна цифра, а четыре типа исходов.

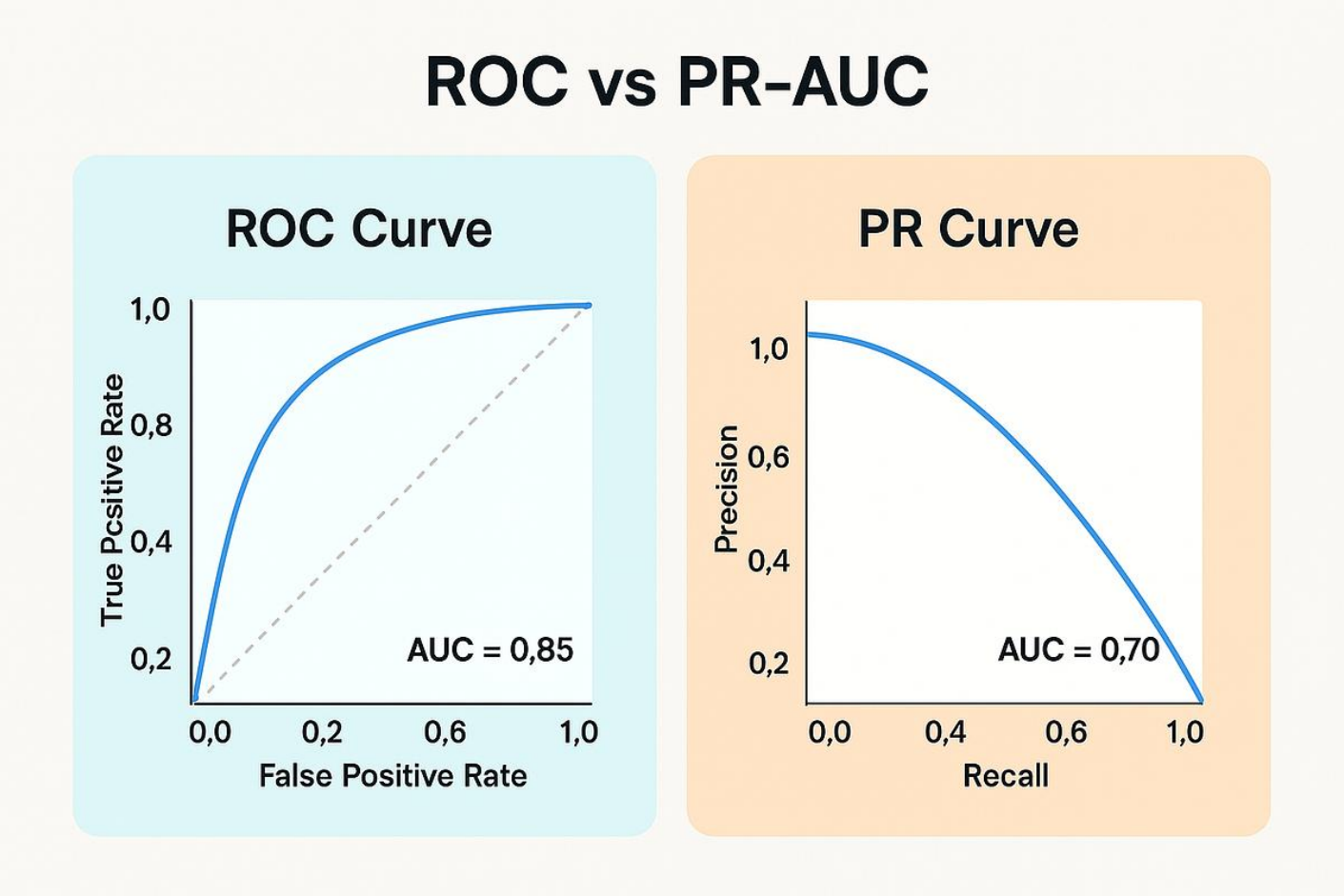


Precision — Из тех, кого назвали выжившими, сколько реально выжили?

Recall — Из всех реально выживших, сколько модель нашла?

ROC-AUC и PR-AUC

Оценивают ranking модели при разных значениях порог.



PR-AUC

Особенно полезна при сильном дисбалансе классов.

ROC-AUC

0.5 ≈ случайное ranking
1.0 = идеальное

Preprocessing: ColumnTransformer

Для деревьев — без scaling. Для SVM и KNN — со StandardScaler.

```python
num = ["Age", "Fare", "SibSp", "Parch"]
cat = ["Pclass", "Sex", "Embarked"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median"))
    ]), num),

    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(handle_unknown="ignore"))
    ]), cat)
])

X_ready = preprocess.fit_transform(X_train)

print(X_train.shape)
print(X_ready.shape)
```

Два режима

preprocess_tree
числа → imputer
категории → imputer + OHE

preprocess_scaled
то же самое + StandardScaler для чисел

OUTPUT

(712, 7) → (712, 12)
4 numeric + 8 one-hot

Baseline

Сначала модель должна победить «ничего не делаем».

```python
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

baseline = DummyClassifier(strategy="most_frequent")

baseline.fit(X_train, y_train)
pred = baseline.predict(X_test)

print("Accuracy:",  accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, zero_division=0))
print("Recall:",    recall_score(y_test, pred))
print("F1:",        f1_score(y_test, pred))
```

Что делает?

Всегда выбирает самый частый класс.

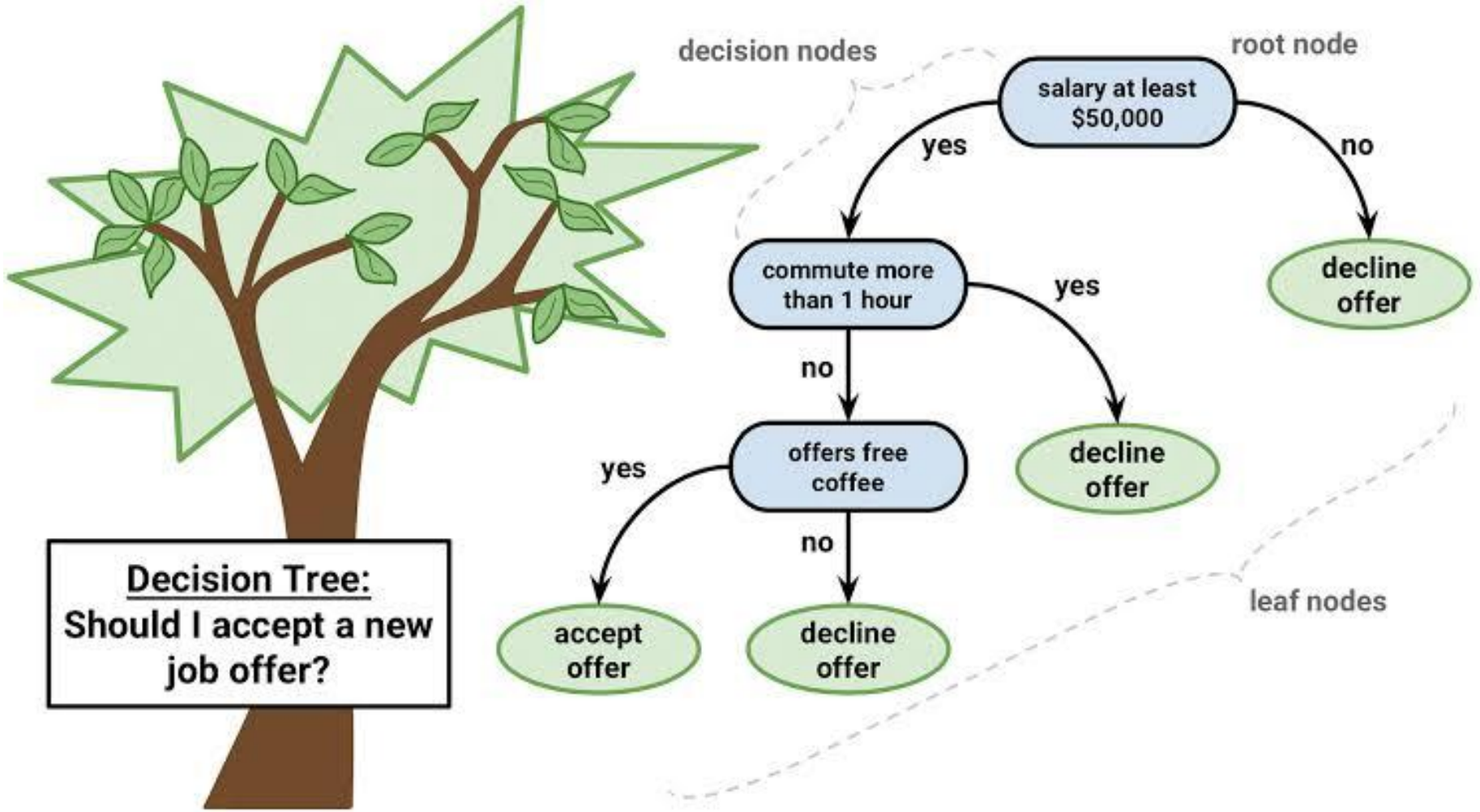
OUTPUT

Accuracy 0.615
Precision 0.000
Recall 0.000
F1 0.000

Если “умная” модель ≈ baseline — проблема не в leaderboard.

Decision Tree

Цепочка простых вопросов по одному признаку.



Как дерево выбирает split?

Ищем разбиение, после которого дочерние узлы становятся «чище».

$$\text{Gini}(S) = 1 - \Sigma\, p_k^2$$

0 → чистый узел

$$\text{Entropy}(S) = -\Sigma\, p_k \log_2 p_k$$

выше → больше неопределённости

Практически Gini и Entropy часто дают близкие деревья.

Глубина и ограничения дерева обычно важнее выбора между ними.

DecisionTreeClassifier

Регуляризуем структуру сразу, а не после переобучения.

```python
from sklearn.tree import DecisionTreeClassifier

tree = Pipeline([
    ("prep", preprocess_tree),
    ("model", DecisionTreeClassifier(
        max_depth=4,
        min_samples_leaf=5,
        random_state=42
    ))
])

tree.fit(X_train, y_train)
pred = tree.predict(X_test)
```

max_depth максимальная глубина

min_samples_leaf минимум объектов в листе

OUTPUT

Accuracy 0.777
Precision 0.809
Recall 0.551
F1 0.655

Дерево умеет идеально выучить train

Нулевая train-ошибка не означает хорошее обобщение.



Глубокое дерево

«Я запомнил каждого пассажира»

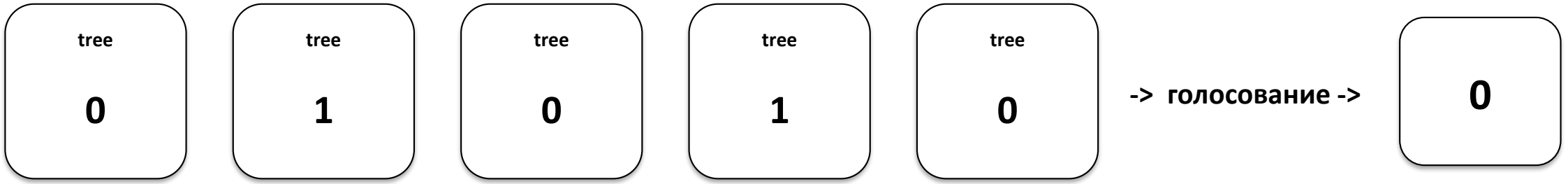
Ограниченное дерево

Меньше вариантивности → обычно лучше на новых данных.

Контроль сложности: max_depth · min_samples_leaf · max_leaf_nodes · ccp_alpha

Random Forest

Много разных деревьев → одно голосование.



- Bootstrap: каждое дерево видит свою выборку объектов.
- Random features: в узле рассматривается случайная часть признаков.
- Bagging уменьшает variance одного дерева.

RandomForestClassifier

Сильный и стабильный baseline для табличных данных.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix
)

rf = Pipeline([
    ("prep", preprocess_tree),
    ("model", RandomForestClassifier(
        n_estimators=300,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1
    ))
])

rf.fit(X_train, y_train)
pred = rf.predict(X_test)

print(f"Accuracy:  {accuracy_score(y_test, pred):.3f}")
print(f"Precision: {precision_score(y_test, pred):.3f}")
print(f"Recall:    {recall_score(y_test, pred):.3f}")
print(f"F1:        {f1_score(y_test, pred):.3f}")
print("CM:", confusion_matrix(y_test, pred).tolist())
```

n_estimators — сколько деревьев

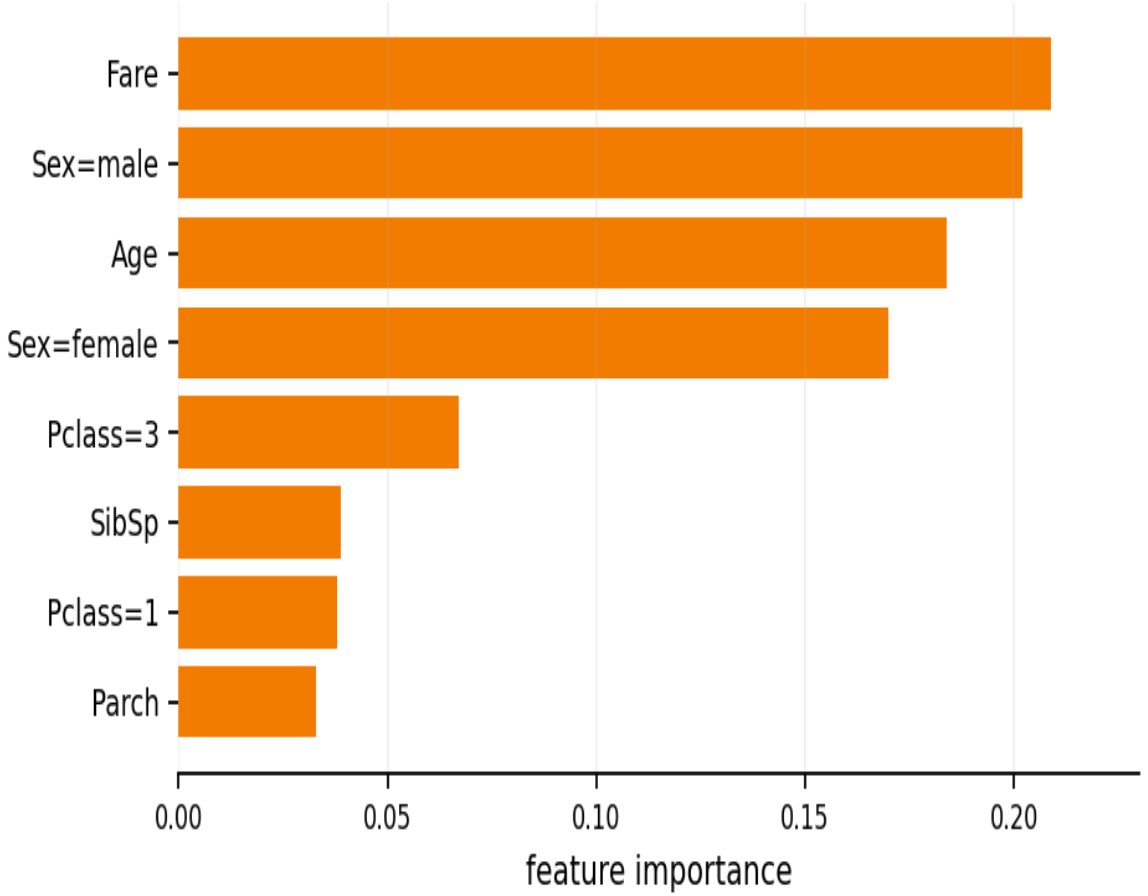
min_samples_leaf — регуляризация деревьев

OUTPUT

Accuracy 0.799
Precision 0.800
Recall 0.638
F1 0.710
CM [[99,11],[25,44]]

Feature importance

Что лес использовал для снижения impurity.



Можно сказать

Какие признаки чаще и сильнее участвовали в полезных split.

Нельзя сказать

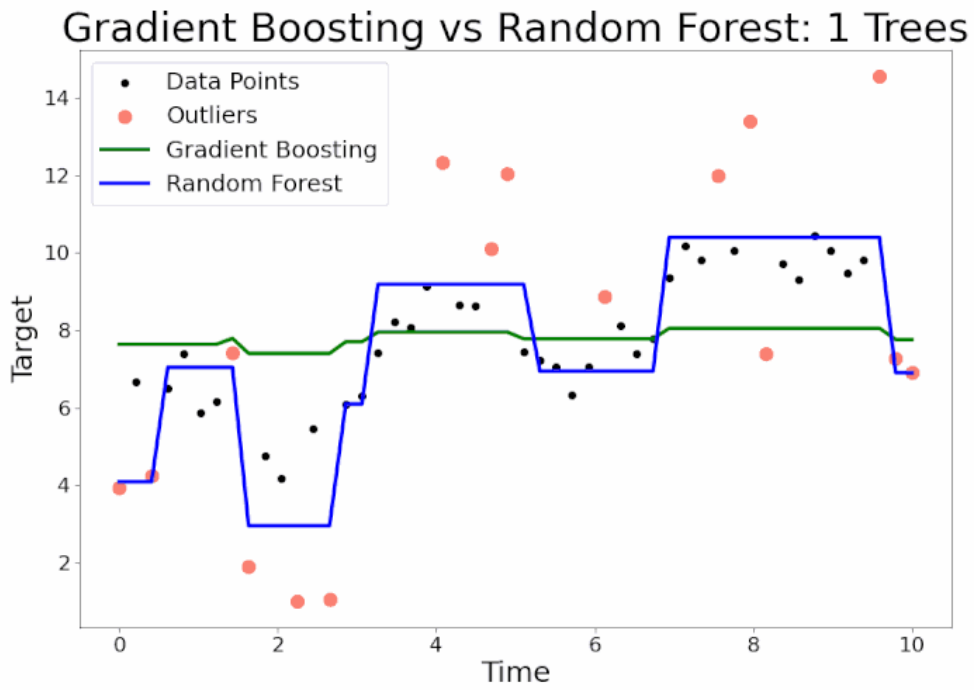
«Этот признак причинно влияет на выживание».

importance ≠ causal inference

Gradient Boosting

Деревья строятся последовательно и исправляют текущие ошибки ансамбля.

Tree 1 + Tree 2 + Tree 3 + … = сильный ансамбль



GradientBoostingClassifier

Сильный табличный алгоритм, чувствительный к сочетанию гиперпараметров.

```python
from sklearn.ensemble import GradientBoostingClassifier

gb = Pipeline([
    ("prep", preprocess_tree),
    ("model", GradientBoostingClassifier(
        n_estimators=150,
        learning_rate=0.05,
        max_depth=2,
        random_state=42
    ))
])

gb.fit(X_train, y_train)
pred_gb = gb.predict(X_test)

```

learning_rate ↓

обучение осторожнее, деревьев нужно больше

max_depth ↑

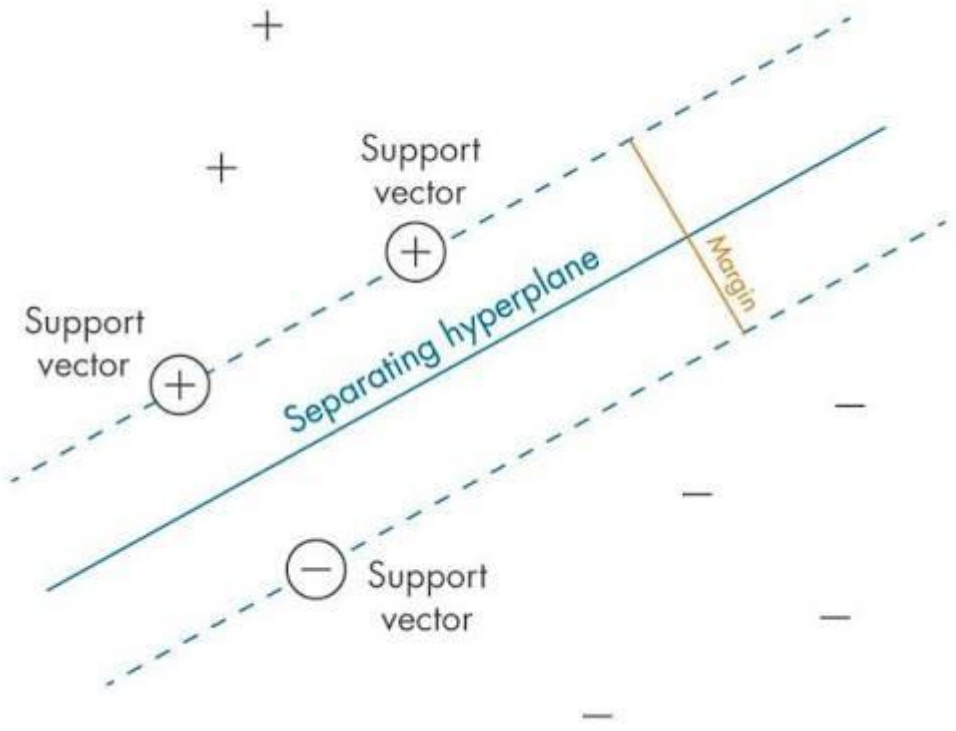
сложнее базовые деревья, выше риск overfit

OUTPUT

Accuracy 0.788
Precision 0.816
Recall 0.580
F1 0.678
ROC-AUC 0.823

SVM

Ищем разделяющую границу с максимальным margin.



Формула для линейного SVM:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)),$$

где:

- $\mathbf{w}$ — вектор весов,
- b — смещение,
- C — параметр регуляризации,
- y_i — метка класса (± 1),
- $\mathbf{x}_i$ — вектор признаков.

C

Большой C сильнее штрафует ошибки → граница сильнее подгоняется под train.

Kernel

RBF позволяет строить нелинейную границу.

Scaling

Критически важен: SVM зависит от геометрии пространства.

SVC

Здесь используем preprocess_scaled: без scaling Fare перетянет геометрию.

```python
from sklearn.svm import SVC

svm = Pipeline([
    ("prep", preprocess_scaled),
    ("model", SVC(
        C=1.0,
        kernel="rbf",
        gamma="scale",
        probability=True
    ))
])

svm.fit(X_train, y_train)
pred_svm = svm.predict(X_test)

```

C ↑ меньше регуляризации

gamma ↑ граница локальнее и сложнее

OUTPUT

Accuracy 0.816
Precision 0.821
Recall 0.667
F1 0.736
ROC-AUC 0.846

KNN

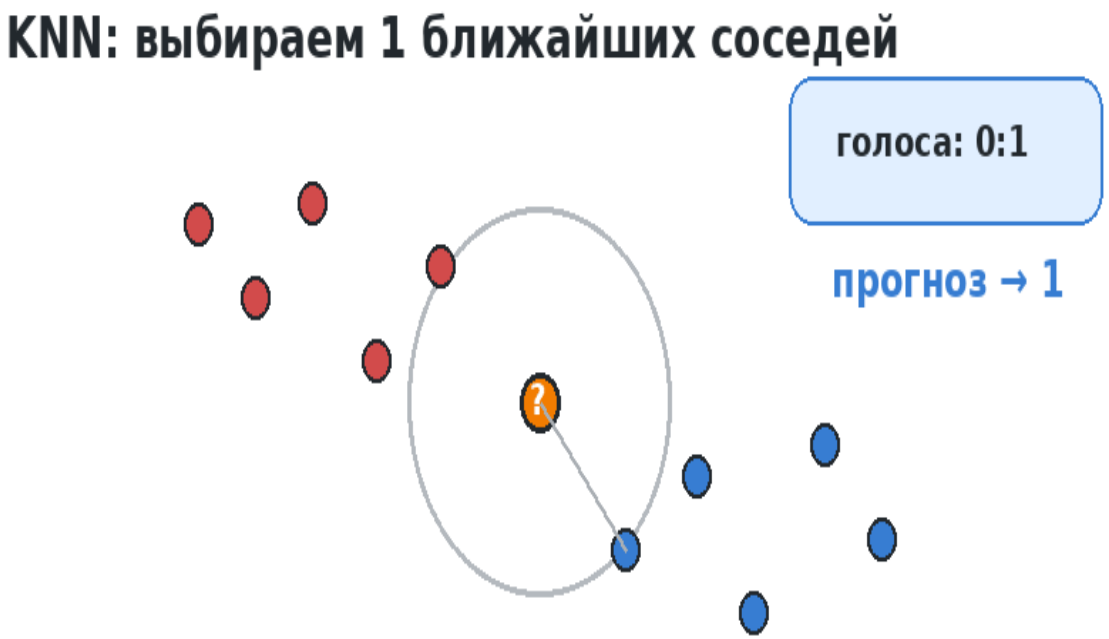
Для нового объекта ищем k ближайших соседей и голосуем по их классам.

k маленькое

очень локально → высокий variance

k большое

граница гладкая → выше bias



Обучение почти отсутствует; predict дороже.

Почему KNN без scaling ломается

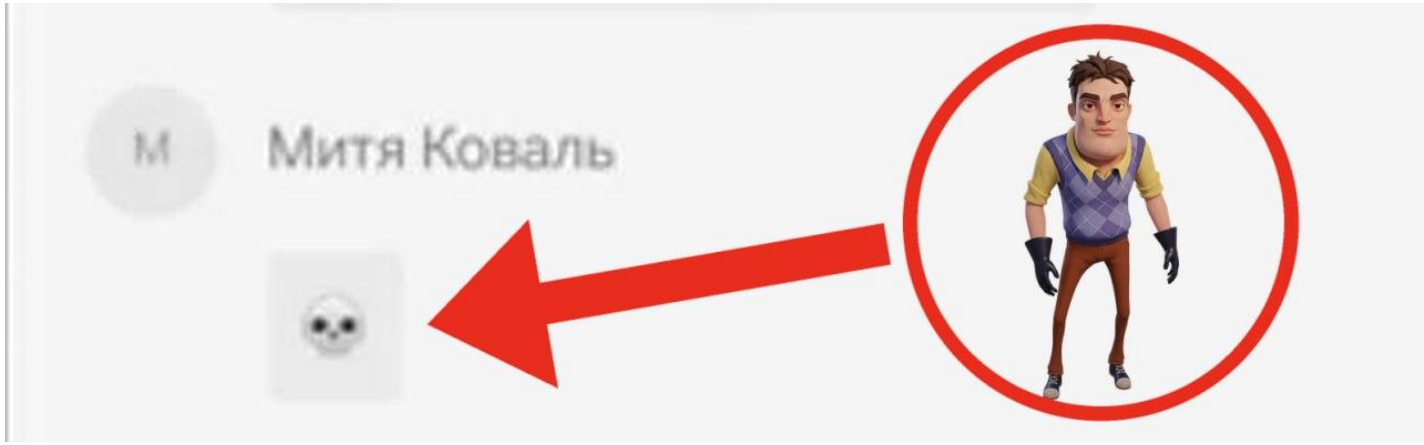
Расстояние считает числа — ему всё равно, что одно число годы, а другое стоимость билета.

До scaling

Fare численно доминирует над Age.

После StandardScaler

Признаки приведены к сопоставимому масштабу.



То же объяснение scaling важно и для SVM.

KNeighborsClassifier

k — главный гиперпараметр; подбираем его по validation / CV, не по test.

```python
from sklearn.neighbors import KNeighborsClassifier

knn = Pipeline([
    ("prep", preprocess_scaled),
    ("model", KNeighborsClassifier(
        n_neighbors=7,
        weights="distance"
    ))
])

knn.fit(X_train, y_train)
pred_knn = knn.predict(X_test)

```

n_neighbors сколько соседей голосует

weights="distance" ближние соседи весят больше

OUTPUT

Accuracy 0.810
Precision 0.769
Recall 0.725
F1 0.746
ROC-AUC 0.854

Одна функция оценки

Fair comparison = одинаковый split и одинаковые метрики.

```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score
)

def evaluate(name, model):
    pred = model.predict(X_test)
    proba = model.predict_proba(X_test)[:, 1]
    return {
        "model": name,
        "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred),
        "recall": recall_score(y_test, pred),
        "f1": f1_score(y_test, pred),
        "roc_auc": roc_auc_score(y_test, proba)
    }

```

Правила

- Один и тот же test для всех моделей.
- F1 — основная метрика лабораторной.
- Confusion matrix анализируем отдельно.

OUTPUT

model name
accuracy 0.799
precision 0.800
recall 0.638
f1 0.710
roc_auc 0.838

Обучаем все пять моделей

Один словарь — один цикл — единый эксперимент.

```python
models = {
    "Decision Tree": tree,
    "Random Forest": rf,
    "Gradient Boosting": gb,
    "SVM": svm,
    "KNN": knn,
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    results.append(evaluate(name, model))

results = (pd.DataFrame(results)
             .sort_values("f1", ascending=False))

```

Ranking по F1

1	**KNN**	**0.746**
2	SVM	**0.736**
3	Random Forest	**0.710**
4	Gradient Boosting	**0.678**
5	Decision Tree	**0.655**

Лучшая одна цифра ≠ автоматически лучшая модель.

Итоговое сравнение

Один split: random_state=42, stratify=y. Реальные результаты кода на Titanic.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy baseline	0.615	0.000	0.000	0.000	0.500
Decision Tree	0.777	0.809	0.551	0.655	0.818
Random Forest	0.799	0.800	0.638	0.710	0.838
Gradient Boosting	0.788	0.816	0.580	0.678	0.823
SVM	0.816	0.821	0.667	0.736	0.846
KNN	**0.810**	**0.769**	**0.725**	**0.746**	**0.854**

Смотрите на баланс Precision/Recall, стабильность и ошибки — не только на максимальный F1.

Пять моделей — пять inductive biases

Нельзя сказать «Random Forest всегда лучше KNN».

Модель	Идея	Scaling	Интерпретация	Когда думать о ней
Decision Tree	правила / пороги	нет	высокая	объяснять решения
Random Forest	ансамбль деревьев	нет	средняя	сильный baseline
Gradient Boosting	последовательный ансамбль	нет	средняя	качество на таблицах
SVM	margin / kernel	да	низкая–средняя	малые/средние выборки
KNN	локальные соседи	да	низкая	простая локальная структура

Модель выбирают под данные, стоимость ошибок и требования к объяснимости / скорости.

